# Chapitre 7 :
# OCaml

## Table des matières


N.1 Introduction : *Particularités d'Ocaml* . . . . . 2

**I Expressions et déclarations . . . . . 3**

I.1 Syntaxe pour lier un identifiant à une expression . . . . . 3

I.2 Expressions numériques . . . . . 3

I.3 Expressions booléennes . . . . . 4

I.4 Expressions conditionnelles . . . . . 4

I.5 Expressions fonctionnelles . . . . . 5

**II Traits impératifs de Ocaml . . . . . 6**

II.1 Instructions . . . . . 6

II.2 Les références . . . . . 8

**III Types . . . . . 8**

III.1 Type `unit` . . . . . 8

III.2 Type `int` . . . . . 8

III.3 Type `float` . . . . . 8

III.4 Type `char` et `string` . . . . . 8

III.5 Types génériques . . . . . 9

III.6 Types « flèches » (ou types fonctionnels) . . . . . 10

III.7 Type option . . . . . 10

III.8 Type exn . . . . . 10

III.9 Tuples = Types produit . . . . . 11

III.10 Listes . . . . . 11

III.11 Tableaux . . . . . 12

**IV Types construits . . . . . 13**

IV.1 Abréviations de type . . . . . 13

IV.2 Enregistrements . . . . . 13

IV.3 Types sommes . . . . . 13

**V Filtrages . . . . . 14**

## N.1 Introduction : *Particularités d'Ocaml*

### *i* Le typage en OCaml

OCaml dispose d'un typage **fort**, **statique** et **inféré**

- Un typage **fort** signifie que que le type est détecté à la compilation.
- Un typage **statique** qu'une « variable » ne peut pas changer implicitement de type.
- Un typage **inféré** signifie que le compilateur déduit les types des objets à l'aide du contexte.

Ces trois propriétés permettent d'assurer qu'un code OCaml qui compile ne comporte aucune erreur de type.

### *i* Paradigme de programmation

OCaml est un langage **fonctionnel**. Ainsi, les fonctions sont des objets comme les autres et un programme est l'application de fonctions sur des objets.

Il diffère ainsi du paradigme **impératif** qui consiste en une succession de modifications de l'état de la mémoire de la machine.

### *i* Informations diverses

OCaml est un langage **compilé**

La gestion de la mémoire est automatique par un **garbage collector**

Ocaml est un langage **strict**, les arguments d'une fonctions sont alors évalués avant son appel.
(Ce n'est pas le cas par exemple du Haskell qui va évaluer les arguments uniquement lorsqu'ils sont nécessaires).

Ocaml est un langage **impur**, il es alors possible de modifier la valeur d'une « variable » (toujours en opposition au Haskell)

# I Expressions et déclarations

En OCaml, tout est une expression ou une phrase. Une phrase étant la liaison d'un identifiant à une expression.
Par exemple, `let _ = f x y` est une phrase, qui lie l'expression `f x y` à l'identifiant `_`.

## I.1 Syntaxe pour lier un identifiant à une expression

**Syntaxe**

let <id> = <expression>

<id> n'est pas une variable, c'est un nom qui permet de référer à <expression>

**Syntaxe : liaison locale et multiple**

let <id> = <$e_1$> in <$e_2$>
let <$id_1$> = <$e_1$> and let <$id_2$> = <$e_2$>

## I.2 Expressions numériques

Tous les entiers sont des expressions de type `int` et si $e_1$ et $e_2$ sont des expressions qui s'évaluent en un objet de type `int`, alors les expressions
$e_1 + e_2$ ; $(-e_1)$ ; $e_1 - e_2$ ; $e_1 * e_2$ ; $e_1$ / $e_2$ ; $e_1 \bmod e_2$ ; abs $e_1$ sont également des expressions de type `int`.

**Avertissement**

Pour calculer l'opposé d'un nombre il faut faire attention aux parenthèses pour ne pas tromper l'analyseur syntaxique.
Par exemple, `Array.make n - 1` ne serait pas compris comme `Array.make n (-1)` mais comme `(Array.make n) - (1)`

Tous les réels représentables sont des expressions de type `float`, et si $e_1$ et $e_2$ s'évaluent en objets de type `float`, alors les expressions
$e_1$ +. $e_2$ ; (-. $e_1$) ; $e_1$ -. $e_2$ ; $e_1$ *. $e_2$ ; $e_1$ /. $e_2$ ; $e_1 ** e_2$ ; abs_float $e_1$ ; exp $s_1$ ; … ; floor $e_1$ ; ceil $e_1$ sont également des expressions de type `float`.

**Avertissement**

`ceil` et `floor` renvoient des flottants, par exemple `floor (-.2.3)` renvoie `-3.`

## I.3 Expressions booléennes

true et false sont des expressions de type `bool` et si $e_1$ et $e_2$ s'évaluent en objets de type `bool` alors les expressions suivantes aussi :
not $e_1$ ; $e_1 \parallel e_2$ ; $e_1 \&\& e_2$

> ⚠ **Avertissement**
>
> || et && sont des opérateurs paressuers.
> Par exemple, `true || f x` ne calculera pas `f x`, de même pour `false && f x`

Si $e_1$ et $e_2$ sont des expressions de même type, alors
$e_1 = e_2$ ; $e_1 < e_2$ ; $e_1 > e_2$ ; $e_1 \leq e_2$ ; $e_1 \geq e_2$ $e_1 <> e_2$ s'évaluent en des objets de type `bool`.

Ceci n'est prudent à faire que sur `int`, `float`, `string`, `int list`, `int array`, etc…
Sur des types fonctionnels il ne faut pas le faire (UB) et sur d'autres types il faut être prudent.

> ⚠ **Avertissement**
>
> $=$ et $\neq$ existent mais testent l'égalité et différence physique et pas de valeurs.
>
> Par exemple, `c = "toto" and d = "toto" in c = d` renvoie true
> `c = "toto" and d = "toto" in c == d` renvoie faux.

## I.4 Expressions conditionnelles

Si $e_1$ est une expression qui s'évalue en `bool` et $e_2$ et $e_3$ sont des expressions **de même type `t`**.
Alors if $e_1$ then $e_2$ else $e_3$ est une expression de type `t`.

> ⚠ **Cas particulier**
>
> Si $e_2$ est de type `unit`, alors on peut écrire simplement if $e_1$ then $e_2$.
> Cela est alors parfaitement équivalent à if $e_1$ then $e_2$ else ().
>
> Ainsi, s'il y a un then sans else, il faut faire particulièrement attention au fait que $e_2$ soit bien de type `unit`

## I.5 Expressions fonctionnelles

### Syntaxe de base

La syntaxe de base pour l'expression d'une fonction est
`fun <id> → <expression>`

Le type de cet objet est inféré à partir des types intervenant dans `<expression>`

On obtient alors une déclaration par `let <nom> = fun <id> → <expression>`

### Un peu de sucre syntaxique

On retrouve plus généralement `let <nom> <id> = <expression>`

Ainsi, l'expresssion `let _ = (fun x → x + 2) 3` est parfaitement valide, au même titre que
`let f x = x + 2 in let _ = f 3`

### *Remarque*

L'expression à droite de → peut elle même être une expression fonctionnelle

Un exemple simple de cette utilisation est `fun x → (fun y → x + y)`

### Une syntaxe très sucrée

On peut plus simplement écrire `fun x y → x + y` mais ce n'est que de la syntaxe, cette fonction reste une fonction prenant un argument et renvoyant une fonction prenant égalment un argument.

Pour simuler le fonctionnement de fonctions à plusieurs arguments, on crée des fonctions qui renvoient des fonctions.
Ce principe s'appelle la **curryfication** ou **currying**, du nom d'Haskell Curry

Ainsi, en Ocaml toutes les fonctions sont curryfiés, ce qui facilite la définition de fonctions partielles. Par exemple, si on note `let f x y = x + y`, alors `f 2` est une déclaration parfaitement valide, qui est la fonction qui prend un argument et lui ajoute 2.

Si on revient au problème de parenthésage précédent, `Array.make n - 1` est compris comme étant `(Array.make n) - (1)` car `Array.make n` est une déclaration parfaitment valide qui est la fonction qui prend un argument et renvoie la liste contenant `n` fois cet argument.

### Le cas des fonctions récursives

Pour les fonctions récursives, il faut ajouter `rec`, la fonction devient alors `let rec <nom> <id> = <expression>` et dans ce cas l'expression peut contenir `<nom>`.

Pour ce qui est des fonctions mutuellement récursives, quand on veut par exemple avoir `f1` et `f2` qui font référence chacune à l'autre.
Cela est défini par `let rec <f1> <arg1> = <exp1> and <f2> <arg2> = <exp2>`

Par exemple, pour chercher la parité d'un entier, on peut (de manière extrêmement maline et efficace)

```ocaml
let rec pair n =
    (n = 0) || (impair (n - 1))
and impair n =
    (n = 1) || (pair (n-1))
```

# II Traits impératifs de Ocaml

## II.1 Instructions

### Syntaxe

Il existe un seul objet littéral de type `unit`, qui est `()`

### Avertissement

Les fonctions de type `unit → 'a` ont besoin de `()` en argument pour les appeler, et il ne faut pas l'oublier.
Par exemple si on a `f () → 'a`, alors `f ()` est un appel de la fonction mais `f` est une fonction.

### Définition

On appelle instruction tout expression de type `unit`

### Les boucles

Par exemple, les boucles sont des instruction.

- La boucle **pour** est définie par

```
for <id> = <debut> to <fin> do
    <instruction>
done
```

  Pour inverser l'ordre, on utilise `<fin> downto <debut>`

- La boucle **tant que** est définie par

```
while <condition> do
    <instruction>
done
```

### Enchaîner des instructions

1. `let _ = <instr1> in let_ = <instr2> in ()`
2. Si `e1` et `e2` sont des instructions, alors `e1;e2` est aussi une instruction.
   `;` est appelé opérateur de séquencement. Il s'agit d'un opérateur binaire qui connecte deux instructions.

`;` peut aussi s'utiliser comme suit : `e_1 ; e_2 ; ... e_n-1 ; e_n`
avec tous les $e_k$ pour $k \in [\![1, n-1]\!]$ de type `unit` et $e_n$ d'un type `'a`.
Alors, toute la déclaration est de type `'a`

### Avertissement

La priorité de l'opérateur `;` est très faible

Par exemple, `if a then b ; c else d` est compris par l'interpréteur comme `(if a then b); c else d`, ce qui renvoie donc une erreur de syntaxe car le else est rattaché nulle part.

De manière encore plus fourbe, `if a then b; c` est compris comme `(if a then b); c`, ce qui peut être syntaxiquement correct si `b` est de type `unit` mais ne fait probablement pas du tout ce qui est attendu.

### Une solution

Pour modifier les priorité, on peut parenthéser avec `(` et `)` ou avec `begin` et `end`.

## II.2 Les références

**Comment manipuler les variables**

- Pour les déclarer, on utilise `let <nom> = ref <expression>`
- Pour les modifier, on utilise `<nom> := <new_expr>`
- Pour accéder à la valeur, on utilise `!<nom>`

Techniquement, une référence est un enregistrement à un seul champ mutable `type 'a ref = {mutable contents : 'a}`. Ainsi, les syntaxes suivantes sont parfaitement équivalentes :

- `let <nom> = {contents = <expression>}` pour la déclaration
- `let <nom>.contents ← <new_expr>` pour la modification
- `<nom>.contents` pour l'accès à la valeur

# III Types

## III.1 Type `unit`

Vu en **II/1**

## III.2 Type `int`

Vu en **I/2**

## III.3 Type `float`

Vu en **I/2**

## III.4 Type `char` et `string`

**Syntaxe**

Les objets de type `char` sont entre `' '`
Les objets de type `string` sont entre `" "`

Ainsi, le code suivant est syntaxiquement incorrect : `let c = 'a' in let s = "a" in a = s`

Pour convertir une chaine `s` à un caractère en le caractère, on utilise `s.[0]`
Pour couvertir un caractère `c` en une chaîne d'un caractère,
on utilise `String.make 1 c`

**Avertissement**

Les objets de type `string` sont **non mutables**

### Syntaxe

Si `s`, `s1` et `s2` sont des objets de type `string`, et si `i` est un objet de type `int` :

| Opération | Effet |
|---|---|
| `s.[i]` | Accès au caractère de `s` en position `i` (En numérotant à partir de 0) |
| `String.length s` | Renvoie le nombre de caractères dans `s` |
| `s1 ^s2` | Renvoie une nouvelle chaîne dont la valeur est la concaténation de `s1` et `s2` Opération en O(\|`s1`\|+\|`s2`\|) |
| `String.sub s début longueur` | Crée une nouvelle chaîne égale à `s.[debut]s.[debut+1]s.[debut+longueur-1]` |

## III.5 Types génériques

### Quand le compilateur met un joker

Si le compilateur ne peut pas inférer le type d'une expression, il utilise un type générique pour identifier ses différentes occurences.

Par exemple, `let f x = x + 2` est de type `int → int` mais `let f x = x` est de type `'a → 'a`

Il y a autant de types génériques générés que de types **différents** qui n'ont pas pu être inféré.
Par exemple, `List.map` est de type `('a → 'b) → 'a list - 'b list`

Les fonctions faisant intervenir dans leur type des types génériques sont dites **polymoprhes**

Un type qui fait intervenir un type générique dans sa définition s'appelle un **type paramétrique**

## III.6 Types « flèches » (ou types fonctionnels)

### Syntaxe

Si `'a` et `'b` sont des types, le type d'une fonction dont les arguments sont de type `'a` et les valeurs sont de type `'b` est `'a → 'b`

Attention, les types `('a → 'b) → 'c` et `'a → ('b → 'c)` sont différents. Par ailleurs, le deuxième est beaucoup plus courant que le premier, au point que l'on omet les parenthèses

## III.7 Type option

### Une valeur, ou pas ?

Les types options sont des types paramétriques : pour tout type `'a`, le type `'a option` est le type option correspondant

Pour « déballer » un objet de type `'a option`, on filtre :

```
match <objet> with
| None → <expression>
| Some x → <expression qui peut faire apparaître x>
```

## III.8 Type exn

### Quand ça merde

Le type `exn` est défini via `exception <Nom> of <type>`.
On peut par exemple définition `exception Vu` ou encore `exception Vu of int`, auquel cas on peut utiliser `raise (Vu 3)`

De nombreuses exceptions sont déjà implémentées, telles que `Division_by_zero`, `Failure of string`, `Invalid_argument of string`, etc…

Il existe egalement `failwith <message>` qui est du sucre syntaxique pour `raise (Failure <message>)`

**Comment rattraper une exception**

```
try
    <expression susceptible de lever une exception>
with
|<exception1> → ...
| ...
```

Attention, le contenu du try et des résultats du filtrage doivent être du même type

## III.9 Tuples = Types produit

**Syntaxe**

Si `'a` et `'b` sont deux types, alors `'a * 'b` est le type des couples dont le premier élément est de type `'a` et le deuxième élément est de type `'b`.

On peut combiner cela pour crer par exemple le type `int * int * float * int list` , à ne pas confondre avec par exemple `(int * int) * float * int list`, un élément du premier serait
`(1, 4, 2.5, [8;9])`, et un exemple du deuxième serait `((1, 4), 2.5, [8;9])`

Pour accéder aux « champs » d'un tuple on peut :
1. Faire un filtrage : `match <tuple> with`
2. Utiliser un let destructurant : `let x, y = t in` où `t` est un couple
3. Une façon qui existe mais à ne pas utiliser (c'est un peu de la merde) : `fst` et `snd` qui renvoient le premier et deuxième élément (mais ne fonctionnent que sur les couples)

## III.10 Listes

cf. poly de syntaxe

**Pleins de trucs au même endroit**

Les listes sont définies par le type paramétrique `'a list`, elles respectent ces conditions :

- Tous les éléments doivent être de type `'a`
- Les éléments sont séparés par des `;`
- Les listes sont non mutables

Même une liste contenant des `refs` n'est pas mutables, c'est une « liste de boîtes » et les « boîtes » peuvent changer de contenu mais c'est toujours les mêmes boîtes, la liste n'as pas changé. Pareil pour une `int list ref`, c'est une boîte qui contient une liste, mais quand on modifie le contenue on remplace juste la liste par une autre liste, la liste n'as pas été modifé

## III.11 Tableaux

cf. poly de syntaxe

**(Encore) pleins de trucs au même endroit**

Les tableaux sont définis par le type paramétrique `'a array`, ils respectent ces conditions :

- Tous les éléments doivent être de type `'a`
- Les éléments sont séparés par des `;`
- Les tableaux sont mutables

En pratique, un tableau est l'adresse de sa premère case, sauf que la taille est stockée avec. En conséquence, `Array.length` est en $O(1)$

**Avertissement**

Comme pour toute structure mutable, il faut faire attention aux éventuels partages de mémoire

**« À n'utiliser qu'après avoir réfléchi beaucoup »**

`Array.of_list` et `Array.to_list` permettent de convertir entre liste et tableau. Il faut faire attention, car si un sujet demande d'utiliser des listes ou des tableaux, c'est pour tester un point précis du Ocaml, changer de type peut-être vu comme un refus de se frotter à quelque chose qu'on ne sait pas faire. S'il est utilisé il faut expliquer pourquoi c'est raisonnable à utiliser et pourquoi ça ne casse pas la complexité

# IV Types construits

**Quésaco ?**

- La défintion d'un nouveau type se fait avec `type <def du type>`
- Tous les constructeurs de type commencent par une majuscule, et seuls eux et les noms de modules en ont la possibilité

## IV.1 Abréviations de type

**Syntaxe**

`type <renommage> = <type qui existe déjà>` permet de renommer un type qui existe déjà, par exemple `type point = float * float * float`

## IV.2 Enregistrements

**Pleins de truc au même endroit (avec des noms !)**

- Ce sont des types produits structurés :
  `let <type> = {<champ1> : <type1> ; ... ; <champk> : <typen> }`
- Pour créer une instance, on utilise `let <id> = {<champ1> = <valeur1> ; ... <champn> = <valeur1>}`
- Pour accéder à la valeur d'un champ, on utilise `<id>.<champ>`
- Par défaut, les champs ne sont pas mutables, mais ils peuvent l'être en rajoutant `mutable` devant le champ correspondant à la définition du type. Pour le modifier on utilise alors `<id>.<champ> ← <nouvelle valeur>`

## IV.3 Types sommes

### Énumération fini (ou à paramètres)

`type <nom> = <Constructeur1> | ... | <Constructeurk>`

Par exemple, `type couleur = Trefle | Pique | Carreau | Coeur`

Pour un type à paramètre, on utilise :
`type <nom> = <Constructeur1> of <type1> | ... | <Constructeurk> of <typek>`
ou chaque déclarateur de type est facultatif. On peut d'ailleurs paramétrer certains constructeurs par un type générique à condition que ce type soit dans le nom du type construit

Par exemple `type 'a ensemble = Vide | Signleton of 'a | Couple of 'a * 'a`

### L'intropection même en info

On peut également créer des types énumérés récursifs sans syntaxe partiuclière. Par exemple : `type 'a arbre = Vide | Feuille of 'a | Noeud of 'a * 'a arbre list`

# V Filtrages

Blabla on sait comment ça fonctionne.

### Les motifs ça a quelle forme ?

- _ → filtre tout
- Constante (ex : “1”, 14, …) → ne filtre que lui même (teste l’égalité)
- Identificateur → filtre tout mais en faisant une liaison entre l’expression et l’id (en masquant les potentielles expression déjà liées à id)
- `[motif1 ; motif2 ; … ; motifk]` → fonctionnement explicite
- `motif1::motif2` → filtre une liste en filtrant la tête avec `motif1` et la queue avec `motif2`
- `(motif1, … , motifk)` → filtres sur un type
- `{<champ1> = moitif1 ; … ; <champk> = motifk}` → filtre sur un enregistrement
- `<Constructeur> motif` → filtre les objets du type construit correpondant en filtrant « l’argument » avec le motif

Pour utiliser un `motif1 | motif2 | … →`, il faut que tous les motifs utilisent exactement les mêmes identifiants. Par exemple, `|0 | 1 → true` ou encore `| Feuille x | Noeud (x,_) → false` sont valides (pour le deuxième si les deux sont de même type) mais `|Feuille x | Noeud (y,l) → false` ne l’est pas

Pour ajouter une condition booléenne à un filtrage, on utilise :
`|motif when <condition> → <expression>`. On dit alors que le cas est « gardé » par la condition